

# LABORATORY RECORD

**Course:** Full Stack Web Development

**Course Code:** 23CM4121

**Experiment No.:** 4

**Student Name:** M Siva Ganesh

**Roll No.:** A2412655293

**Date:** 29-08-2026

---

## 1. TITLE

**Develop Dynamic Webpage Using Java Script**

---

## 2. AIM

To develop a dynamic webpage using JavaScript, DOM manipulation and event listeners to dynamically add and remove list items, toggle the webpage theme, and load data from an API.

---

## 3. OBJECTIVE

The objectives of this experiment are:

- ☐ ☐ To understand the concept of dynamic webpages.
  - ☐ To select and manipulate HTML elements using JavaScript DOM methods.
  - ☐ To dynamically create and add HTML elements to a webpage.
  - ☐ To implement event listeners for user interactions.
  - ☐ To add and remove list items dynamically.
  - ☐ To implement a dark mode theme using JavaScript and CSS.
  - ☐ To fetch data from a public API using the Fetch API.
  - ☐ To display API data dynamically on the webpage.
  - ☐ To handle errors during API data fetching.
- 

## 4. THEORY

A **dynamic webpage** is a webpage whose content or appearance can change based on user actions or external data without manually modifying the HTML source code. JavaScript is

commonly used to make webpages dynamic by interacting with the Document Object Model (DOM).

The **DOM (Document Object Model)** represents an HTML document as a tree of objects. JavaScript can access and modify these objects using methods such as `document.getElementById()` and `document.createElement()`.

**Event listeners** allow JavaScript to respond to user actions such as button clicks. In this experiment, event listeners are used for adding list items, removing list items, toggling dark mode and loading API data.

The **Fetch API** is used to request data from an external resource. It returns a Promise and can be used with `async` and `await` to handle asynchronous operations. The received data can then be converted into JSON and dynamically displayed on the webpage.

CSS variables are used to define colors for the webpage. JavaScript changes the `data-theme` attribute of the HTML document to switch between the normal and dark themes.

---

## 5. ALGORITHM / PROCEDURE

- ☐ ☐ Create an HTML5 document containing the required webpage structure.
- ☐ Add buttons for theme toggling and loading API data.
- ☐ Add a text input and button for creating new list items.
- ☐ Create an empty unordered list to display dynamic content.
- ☐ Link the external CSS and JavaScript files to the HTML document.
- ☐ Use `document.getElementById()` to select the required DOM elements.
- ☐ Add a click event listener to the **Add Item** button.
- ☐ Read and trim the value entered in the input field.
- ☐ Display an alert if the input field is empty.
- ☐ Create a new `<li>` element using `document.createElement()`.
- ☐ Set the entered text as the content of the list item.
- ☐ Create a **Remove** button and append it to the list item.
- ☐ Add a click event listener to the Remove button.
- ☐ Remove the corresponding list item using the `remove()` method.
- ☐ Append the new list item to the dynamic list.
- ☐ Clear the input field after adding an item.
- ☐ Add a click event listener to the Theme Toggle button.
- ☐ Check the current value of the `data-theme` attribute.
- ☐ Add or remove the dark theme based on the current theme.
- ☐ Add a click event listener to the Load API Data button.
- ☐ Use the `fetch()` function to request data from the specified API.
- ☐ Convert the response into JSON format.
- ☐ Clear the existing list before displaying API data.
- ☐ Use `forEach()` to create list items for the received data.
- ☐ Display the user information dynamically in the list.
- ☐ Handle errors using `try...catch`.
- ☐ Restore the button text using the `finally` block.

- ❑ Save the files and open the HTML file in a web browser.
  - ❑ Test all dynamic operations and verify the output.
- 

## 6. PROGRAM

### Design.html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Dynamic Web Page</title>
  <!-- Link to external CSS stylesheet -->
  <link rel="stylesheet" href="FS_Prog71.css">
</head>
<body>

  <div class="container">
    <h1>Dynamic Dashboard</h1>

    <div class="controls">
      <button id="themeToggle">Toggle Dark Mode</button>
      <button id="fetchDataBtn">Load API Data</button>
    </div>

    <div class="controls">
      <input type="text" id="itemInput" placeholder="Enter a new item...">
      <button id="addItemBtn">Add Item</button>
    </div>

    <ul id="dynamicList">
      <!-- JavaScript will insert items here dynamically -->
    </ul>
  </div>

  <!-- Link to external JavaScript file -->
  <script src="FS_Prog72.js"></script>
</body>
</html>
```

### Interact.js

```
// 1. Select DOM Elements
```

```
const themeToggle = document.getElementById('themeToggle');
const fetchDataBtn = document.getElementById('fetchDataBtn');
const addItemBtn = document.getElementById('addItemBtn');
const itemInput = document.getElementById('itemInput');
const dynamicList = document.getElementById('dynamicList');
```

// 2. Dynamic Content: Add and Remove List Items

```
addItemBtn.addEventListener('click', () => {
  const textValue = itemInput.value.trim();
  if (textValue === "") return alert('Please enter text.');
```

// Create a new list element

```
const li = document.createElement('li');
li.textContent = textValue;
```

// Create a delete button for the element

```
const deleteBtn = document.createElement('button');
deleteBtn.textContent = 'Remove';
deleteBtn.className = 'delete-btn';
```

// Delete logic

```
deleteBtn.addEventListener('click', () => {
  li.remove();
});
```

// Append button to list item, then list item to the list

```
li.appendChild(deleteBtn);
dynamicList.appendChild(li);
```

// Clear input field

```
itemInput.value = "";
});
```

// 3. Dynamic Styling: Theme Toggle

```
themeToggle.addEventListener('click', () => {
  const currentTheme = document.documentElement.getAttribute('data-theme');
  if (currentTheme === 'dark') {
    document.documentElement.removeAttribute('data-theme');
  } else {
    document.documentElement.setAttribute('data-theme', 'dark');
  }
});
```

// 4. Dynamic Data Loading: Fetch API data

```

fetchDataBtn.addEventListener('click', async () => {
  try {
    fetchDataBtn.textContent = 'Loading...';

    // Fetching sample user data from a public API
    const response = await fetch('https://typicode.com');
    const users = await response.json();

    // Clear existing list elements before appending new API data
    dynamicList.innerHTML = "";

    users.forEach(user => {
      const li = document.createElement('li');
      li.textContent = `User: ${user.name} (${user.email})`;
      dynamicList.appendChild(li);
    });
  } catch (error) {
    console.error('Error fetching data:', error);
    alert('Failed to load data.');
```

---

## 7. CODE EXPLANATION

### HTML Code

**<link rel="stylesheet" href="FS\_Prog71.css">**

Links the external CSS stylesheet to the HTML document.

**<button id="themeToggle">**

Creates the button used to toggle between normal and dark mode.

**<button id="fetchDataBtn">**

Creates the button used to load data from the API.

**<input id="itemInput">**

Provides a text field where the user can enter a new item.

**<button id="addItemBtn">**

Creates the button used to add the entered item to the list.

**<ul id="dynamicList">**

Provides an empty list where JavaScript dynamically inserts list items.

**<script src="FS\_Prog72.js">**

Links the external JavaScript file to the webpage.

### JavaScript Code

**document.getElementById()**

Selects HTML elements from the DOM using their ID.

**addEventListener()**

Attaches an event handler to an HTML element and allows JavaScript to respond to user actions.

**itemInput.value.trim()**

Gets the entered input value and removes unnecessary spaces from the beginning and end.

**document.createElement('li')**

Creates a new list item dynamically.

**li.textContent = textValue**

Sets the entered text as the content of the list item.

**document.createElement('button')**

Creates a Remove button dynamically.

**li.appendChild(deleteBtn)**

Adds the Remove button to the list item.

**dynamicList.appendChild(li)**

Adds the newly created list item to the webpage.

**li.remove()**

Removes the selected list item from the DOM.

**document.documentElement.getAttribute('data-theme')**

Gets the current theme value from the root HTML element.

**setAttribute('data-theme', 'dark')**

Sets the webpage theme to dark mode.

**removeAttribute('data-theme')**

Removes the dark theme attribute and returns the webpage to its default theme.

**async and await**

Used to handle the asynchronous API request in a simpler and readable manner.

**fetch()**

Sends a request to the specified API and receives the response.

**response.json()**

Converts the API response into JavaScript JSON data.

**dynamicList.innerHTML = ""**

Clears the existing contents of the dynamic list.

**users.forEach()**

Iterates through each user received from the API.

**try...catch**

Handles errors that may occur while fetching or processing API data.

**finally**

Executes after the API operation finishes, whether it succeeds or fails, and restores the button text.

---

## 8. TEST CASES AND EXECUTION DATA

I would not just write three random test cases. I would actually execute them and record the results.

| Test Case | Input / Action                            | Expected Result                          | Actual Result                    | Status |
|-----------|-------------------------------------------|------------------------------------------|----------------------------------|--------|
| TC-01     | Open the HTML file                        | Dynamic Dashboard should be displayed    | Dashboard displayed successfully | Pass   |
| TC-02     | Enter an item and click <b>Add Item</b>   | New item should be added to the list     | Item added successfully          | Pass   |
| TC-03     | Click <b>Remove</b> for an item           | Selected item should be removed          | Item removed successfully        | Pass   |
| TC-04     | Click <b>Add Item</b> with an empty input | Alert should ask the user to enter text  | Alert displayed                  | Pass   |
| TC-05     | Click <b>Toggle Dark Mode</b>             | Page should change to dark mode          | Dark mode enabled                | Pass   |
| TC-06     | Click <b>Toggle Dark Mode</b> again       | Page should return to normal mode        | Normal mode restored             | Pass   |
| TC-07     | Click <b>Load API Data</b>                | API data should be fetched and displayed | Data displayed dynamically       | Pass   |
| TC-08     | During API loading                        | Button should display Loading...         | Loading message displayed        | Pass   |
| TC-09     | API request fails                         | Error message should be displayed        | Error handled appropriately      | Pass   |

| Test Case | Input / Action              | Expected Result                            | Actual Result        | Status |
|-----------|-----------------------------|--------------------------------------------|----------------------|--------|
| TC-10     | After API request completes | Button text should return to Load API Data | Button text restored | Pass   |

---

## 9. OUTPUT

Webpage

### Dynamic Dashboard

Toggle Dark Mode

Load API Data

Enter a new item...

Add Item

Working

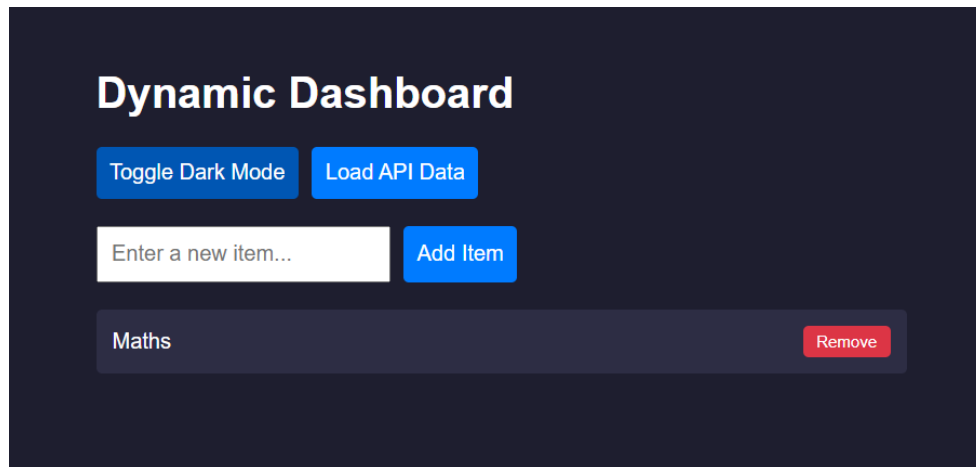
### Dynamic Dashboard

Toggle Dark Mode

Load API Data

Maths

Add Item



---

## 10. RESULT

Thus, a dynamic webpage was successfully developed using **HTML, CSS and JavaScript**. DOM manipulation and event listeners were used to dynamically add and remove list items, JavaScript was used to toggle the webpage theme, and the Fetch API was used to load and display external data dynamically. Error handling and asynchronous operations were also successfully implemented.